

DS1_to_A2

July 2, 2026

0.1 ## Adaptación del Dataset - DS1 a las características del Artículo 2 - A2

1 1. DESCARGA DEL DATASET (DS1) DESDE LA URL OFICIAL

```
[1]: # 1.1 Descargar datos
import urllib.request
import zipfile
import io
import pandas as pd

print("DESCARGA DEL DATASET")
url = "https://archive.ics.uci.edu/static/public/321/
electricityloadaddiagrams20112014.zip"
with urllib.request.urlopen(url) as response:
    zip_data = io.BytesIO(response.read())
with zipfile.ZipFile(zip_data) as zip_file:
    with zip_file.open('LD2011_2014.txt') as file:
        df_raw = pd.read_csv(file, sep=';', decimal=',', low_memory=False)

# 1.2 Procesamiento básico
df_raw.rename(columns={df_raw.columns[0]: 'datetime'}, inplace=True)
df_raw['datetime'] = pd.to_datetime(df_raw['datetime'], dayfirst=False)
df_raw.set_index('datetime', inplace=True)

print(f" Dataset cargado. Shape: {df_raw.shape} (filas, columnas)")
print(f" Rango de fechas: {df_raw.index.min()} -> {df_raw.index.max()}")
print(f" Número total de clientes: {df_raw.shape[1]}\n")

display(df_raw)
```

DESCARGA DEL DATASET

Dataset cargado. Shape: (140256, 370) (filas, columnas)

Rango de fechas: 2011-01-01 00:15:00 -> 2015-01-01 00:00:00

Número total de clientes: 370

		MT_001	MT_002	MT_003	MT_004	MT_005 \
datetime						
2011-01-01	00:15:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	00:30:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	00:45:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	01:00:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	01:15:00	0.000000	0.000000	0.000000	0.000000	0.000000
...						
2014-12-31	23:00:00	2.538071	22.048364	1.737619	150.406504	85.365854
2014-12-31	23:15:00	2.538071	21.337127	1.737619	166.666667	81.707317
2014-12-31	23:30:00	2.538071	20.625889	1.737619	162.601626	82.926829
2014-12-31	23:45:00	1.269036	21.337127	1.737619	166.666667	85.365854
2015-01-01	00:00:00	2.538071	19.914651	1.737619	178.861789	84.146341

		MT_006	MT_007	MT_008	MT_009	MT_010 \
datetime						
2011-01-01	00:15:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	00:30:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	00:45:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	01:00:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	01:15:00	0.000000	0.000000	0.000000	0.000000	0.000000
...						
2014-12-31	23:00:00	303.571429	11.305822	282.828283	68.181818	72.043011
2014-12-31	23:15:00	324.404762	11.305822	252.525253	64.685315	72.043011
2014-12-31	23:30:00	318.452381	10.175240	242.424242	61.188811	74.193548
2014-12-31	23:45:00	285.714286	10.175240	225.589226	64.685315	72.043011
2015-01-01	00:00:00	279.761905	10.175240	249.158249	62.937063	69.892473

		...	MT_361	MT_362	MT_363	MT_364 \
datetime						
2011-01-01	00:15:00	...	0.000000	0.0	0.000000	0.000000
2011-01-01	00:30:00	...	0.000000	0.0	0.000000	0.000000
2011-01-01	00:45:00	...	0.000000	0.0	0.000000	0.000000
2011-01-01	01:00:00	...	0.000000	0.0	0.000000	0.000000
2011-01-01	01:15:00	...	0.000000	0.0	0.000000	0.000000
...						
2014-12-31	23:00:00	...	276.945039	28200.0	1616.033755	1363.636364
2014-12-31	23:15:00	...	279.800143	28300.0	1569.620253	1340.909091
2014-12-31	23:30:00	...	284.796574	27800.0	1556.962025	1318.181818
2014-12-31	23:45:00	...	246.252677	28000.0	1443.037975	909.090909
2015-01-01	00:00:00	...	188.436831	27800.0	1409.282700	954.545455

		MT_365	MT_366	MT_367	MT_368	MT_369 \
datetime						
2011-01-01	00:15:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	00:30:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	00:45:00	0.000000	0.000000	0.000000	0.000000	0.000000
2011-01-01	01:00:00	0.000000	0.000000	0.000000	0.000000	0.000000

2011-01-01 01:15:00	0.000000	0.000000	0.000000	0.000000	0.000000
...	...	...	...	...	...
2014-12-31 23:00:00	29.986962	5.851375	697.102722	176.961603	651.026393
2014-12-31 23:15:00	29.986962	9.947338	671.641791	168.614357	669.354839
2014-12-31 23:30:00	27.379400	9.362200	670.763828	153.589316	670.087977
2014-12-31 23:45:00	26.075619	4.095963	664.618086	146.911519	646.627566
2015-01-01 00:00:00	27.379400	4.095963	628.621598	131.886477	673.020528

MT_370

```

datetime
2011-01-01 00:15:00    0.000000
2011-01-01 00:30:00    0.000000
2011-01-01 00:45:00    0.000000
2011-01-01 01:00:00    0.000000
2011-01-01 01:15:00    0.000000
...
2014-12-31 23:00:00  7621.621622
2014-12-31 23:15:00  6702.702703
2014-12-31 23:30:00  6864.864865
2014-12-31 23:45:00  6540.540541
2015-01-01 00:00:00  7135.135135

```

[140256 rows x 370 columns]

2 EXPLORACIÓN DEL DATASET: BÚSQUEDA DEL MEJOR CLIENTE

Según la documentación oficial del dataset `ElectricityLoadDiagrams20112014`:

- Los valores están en **kW** cada **15 minutos**.
- Para convertirlos a **kWh**, hay que dividir cada valor entre 4 (o sumar 4 cuartos de hora para obtener el total de esa hora).
- **Cada columna** (MT_XXX) representa un **cliente/usuario** diferente.
- **Algunos clientes fueron creados después de 2011**, por lo que sus consumos anteriores aparecen como **cero**.
- **Cambios de hora (DST)**:
 - **Marzo** (día de 23 horas): Las mediciones entre la 1:00 y las 2:00 a.m. son **cero**.
 - **Octubre** (día de 25 horas): Las mediciones entre la 1:00 y las 2:00 a.m. **agregan el consumo de dos horas**.

Objetivo de este búsqueda:

Encontrar el cliente MT_XXX que tenga **mayor continuidad temporal** (activo desde el inicio) y **menor proporción de ceros/NaN**, para usarlo como nuestro “cliente único” y así replicar la metodología del artículo de Zhang et al. (2021).

2.1 Resultados esperados

El script generará:

1. Un resumen estadístico de los primeros y últimos clientes.
2. Una lista de **candidatos válidos** (activos desde 2011-01-01 y con menos del 5% de ceros).
3. El nombre del **mejor cliente** (el que tiene el menor porcentaje de ceros).
4. La preparación final de la serie horaria (frecuencia 1h) y la creación de características temporales (hour, dayofweek, month, dayofyear).

```
[2]: # =====  
# 2. EXPLORACIÓN DE CLIENTES (BÚSQUEDA DEL MEJOR CANDIDATO)  
# =====  
print(" Explorando cada cliente para encontrar el más estable...")  
  
# Calculamos estadísticas por columna  
stats = []  
for col in df_raw.columns:  
    serie = df_raw[col]  
  
    # Fecha del primer valor > 0 (momento en que el cliente se activó)  
    first_active = serie[serie > 0].index.min()  
  
    # Porcentaje de ceros  
    zero_ratio = (serie == 0).mean()  
  
    # Porcentaje de NaN (si los hay)  
    nan_ratio = serie.isna().mean()  
  
    # Cantidad total de datos  
    total = len(serie)  
  
    stats.append({  
        'cliente': col,  
        'first_active': first_active,  
        'zero_ratio': zero_ratio,  
        'nan_ratio': nan_ratio,  
        'total_datos': total  
    })  
  
# Convertimos a DataFrame para ordenar y filtrar  
df_stats = pd.DataFrame(stats)  
  
# Mostramos los 5 clientes con menor proporción de ceros (los más "activos")  
print("\n Top 5 clientes con menos ceros (los más estables):")  
print(df_stats.sort_values('zero_ratio').head(5).to_string(index=False))
```

```

# Mostramos los 5 clientes con mayor proporción de ceros (los que se activaron
↳tarde)
print("\n Top 5 clientes con más ceros (activación tardía o nulos):")
print(df_stats.sort_values('zero_ratio', ascending=False).head(5).
↳to_string(index=False))

# =====
# 3. FILTRADO DE CANDIDATOS VÁLIDOS
# =====
# Condiciones:
# 1. Que se hayan activado el primer día o al día siguiente (2011-01-01 o
↳2011-01-02)
# 2. Que tengan menos del 5% de ceros (para descartar los que se activaron
↳tarde)
fecha_limite = pd.Timestamp('2011-01-02')
umbral_cero = 0.05

candidatos = df_stats[
    (df_stats['first_active'] <= fecha_limite) &
    (df_stats['zero_ratio'] < umbral_cero)
]

print(f"\n Clientes candidatos (activos antes de {fecha_limite.date()} y con <
↳{umbral_cero*100}% de ceros:")
print(candidatos.to_string(index=False))

if candidatos.empty:
    # Si no hay candidatos, relajamos el filtro a 2011-01-03
    fecha_limite = pd.Timestamp('2011-01-03')
    candidatos = df_stats[
        (df_stats['first_active'] <= fecha_limite) &
        (df_stats['zero_ratio'] < umbral_cero)
    ]
    print(f"\n No se encontraron candidatos con la fecha límite anterior.
↳Usando {fecha_limite.date()}")
    print(candidatos.to_string(index=False))

# Elegimos el mejor: el que tenga el menor ratio de ceros
mejor_cliente = candidatos.loc[candidatos['zero_ratio'].idxmin(), 'cliente']
zero_ratio_mejor = candidatos.loc[candidatos['zero_ratio'].idxmin(),
↳'zero_ratio']

print(f"\n CLIENTE SELECCIONADO: **{mejor_cliente}**")
print(f"    - Proporción de ceros: {zero_ratio_mejor:.4%}")
print(f"    - Fecha de primera actividad:
↳{df_stats[df_stats['cliente']==mejor_cliente]['first_active'].values[0]}")

```

```
# =====
# 4. EXTRACCIÓN DE LA SERIE DEL CLIENTE ELEGIDO
# =====
serie = df_raw[mejor_cliente]
serie.name = 'consumo'

print(f"\n Extrayendo serie del cliente {mejor_cliente}...")
print(f"    - Registros cada 15 min: {len(serie)}")
print(f"    - Valores NaN: {serie.isna().sum()}")
print(f"    - Valores cero: {(serie == 0).sum()}")

# =====
# 5. REMUESTREO A FRECUENCIA HORARIA (SUMA DE 4 CUARTOS DE HORA)
# =====
# NOTA: Sumamos los 4 cuartos de hora (15 min) para obtener el consumo total
# en kWh durante esa hora. Como el artículo trabaja con potencias (kW),
# usar la suma es equivalente a la potencia media durante esa hora (porque 1
#    ↪ kWh en 1h = 1 kW de media).
serie_hourly = serie.resample('H').sum()

# Eliminamos posibles NaN que puedan aparecer por los cambios de hora (DST)
serie_hourly = serie_hourly.dropna()

print(f"\n Serie convertida a frecuencia horaria:")
print(f"    - Registros horarios: {len(serie_hourly)}")
print(f"    - Rango: {serie_hourly.index.min()} -> {serie_hourly.index.max()}")
```

Explorando cada cliente para encontrar el más estable...

Top 5 clientes con menos ceros (los más estables):

cliente	first_active	zero_ratio	nan_ratio	total_datos
MT_290	2011-01-01 00:15:00	0.0	0.0	140256
MT_292	2011-01-01 00:15:00	0.0	0.0	140256
MT_293	2011-01-01 00:15:00	0.0	0.0	140256
MT_294	2011-01-01 00:15:00	0.0	0.0	140256
MT_296	2011-01-01 00:15:00	0.0	0.0	140256

Top 5 clientes con más ceros (activación tardía o nulos):

cliente	first_active	zero_ratio	nan_ratio	total_datos
MT_178	2014-07-18 00:15:00	0.886379	0.0	140256
MT_133	2014-03-13 16:15:00	0.868127	0.0	140256
MT_181	2014-03-05 00:15:00	0.793977	0.0	140256
MT_109	2014-02-18 00:15:00	0.783061	0.0	140256
MT_116	2014-02-18 00:15:00	0.783054	0.0	140256

Clientes candidatos (activos antes de 2011-01-02 y con < 5.0% de ceros):

cliente	first_active	zero_ratio	nan_ratio	total_datos
MT_124	2011-01-01 00:15:00	0.000086	0.0	140256
MT_156	2011-01-01 00:15:00	0.000000	0.0	140256
MT_158	2011-01-01 00:15:00	0.000000	0.0	140256
MT_159	2011-01-01 00:15:00	0.000000	0.0	140256
MT_161	2011-01-01 00:15:00	0.000128	0.0	140256
MT_162	2011-01-01 00:15:00	0.000321	0.0	140256
MT_163	2011-01-01 00:15:00	0.000150	0.0	140256
MT_166	2011-01-01 00:15:00	0.000000	0.0	140256
MT_168	2011-01-01 00:15:00	0.000000	0.0	140256
MT_169	2011-01-01 00:15:00	0.000000	0.0	140256
MT_171	2011-01-01 00:15:00	0.000000	0.0	140256
MT_172	2011-01-01 00:15:00	0.000000	0.0	140256
MT_174	2011-01-01 00:15:00	0.000684	0.0	140256
MT_175	2011-01-01 00:15:00	0.000684	0.0	140256
MT_176	2011-01-01 00:15:00	0.000000	0.0	140256
MT_180	2011-01-01 00:15:00	0.000000	0.0	140256
MT_182	2011-01-01 00:15:00	0.000000	0.0	140256
MT_183	2011-01-01 00:15:00	0.000000	0.0	140256
MT_187	2011-01-01 00:15:00	0.000100	0.0	140256
MT_188	2011-01-01 00:15:00	0.000684	0.0	140256
MT_189	2011-01-01 00:15:00	0.000121	0.0	140256
MT_190	2011-01-01 00:15:00	0.000150	0.0	140256
MT_191	2011-01-01 00:15:00	0.000128	0.0	140256
MT_192	2011-01-01 00:15:00	0.000406	0.0	140256
MT_193	2011-01-01 00:15:00	0.000221	0.0	140256
MT_194	2011-01-01 00:15:00	0.000171	0.0	140256
MT_195	2011-01-01 00:15:00	0.000143	0.0	140256
MT_196	2011-01-01 00:15:00	0.000157	0.0	140256
MT_197	2011-01-01 00:15:00	0.000143	0.0	140256
MT_198	2011-01-01 00:15:00	0.000171	0.0	140256
MT_199	2011-01-01 00:15:00	0.000135	0.0	140256
MT_200	2011-01-01 00:15:00	0.000114	0.0	140256
MT_201	2011-01-01 00:15:00	0.000200	0.0	140256
MT_202	2011-01-01 00:15:00	0.000121	0.0	140256
MT_203	2011-01-01 00:15:00	0.000763	0.0	140256
MT_204	2011-01-01 00:15:00	0.000100	0.0	140256
MT_205	2011-01-01 00:15:00	0.000799	0.0	140256
MT_206	2011-01-01 00:15:00	0.000114	0.0	140256
MT_207	2011-01-01 00:15:00	0.000221	0.0	140256
MT_208	2011-01-01 00:15:00	0.000157	0.0	140256
MT_209	2011-01-01 00:15:00	0.000827	0.0	140256
MT_210	2011-01-01 00:15:00	0.000107	0.0	140256
MT_211	2011-01-01 00:15:00	0.000164	0.0	140256
MT_212	2011-01-01 00:15:00	0.000100	0.0	140256
MT_213	2011-01-01 00:15:00	0.000349	0.0	140256
MT_214	2011-01-01 00:15:00	0.000171	0.0	140256
MT_215	2011-01-01 00:15:00	0.000107	0.0	140256

MT_216	2011-01-01	00:15:00	0.000207	0.0	140256
MT_217	2011-01-01	00:15:00	0.000121	0.0	140256
MT_218	2011-01-01	00:15:00	0.000178	0.0	140256
MT_219	2011-01-01	00:15:00	0.000585	0.0	140256
MT_220	2011-01-01	00:15:00	0.000107	0.0	140256
MT_221	2011-01-01	00:15:00	0.000114	0.0	140256
MT_222	2011-01-01	00:15:00	0.000100	0.0	140256
MT_225	2011-01-01	00:15:00	0.000000	0.0	140256
MT_226	2011-01-01	00:15:00	0.000392	0.0	140256
MT_227	2011-01-01	00:15:00	0.000128	0.0	140256
MT_228	2011-01-01	00:15:00	0.000086	0.0	140256
MT_229	2011-01-01	00:15:00	0.000898	0.0	140256
MT_230	2011-01-01	00:15:00	0.000086	0.0	140256
MT_231	2011-01-01	00:15:00	0.000000	0.0	140256
MT_232	2011-01-01	00:15:00	0.000000	0.0	140256
MT_233	2011-01-01	00:15:00	0.000000	0.0	140256
MT_234	2011-01-01	00:15:00	0.000000	0.0	140256
MT_235	2011-01-01	00:15:00	0.000000	0.0	140256
MT_236	2011-01-01	00:15:00	0.000684	0.0	140256
MT_237	2011-01-01	00:15:00	0.000000	0.0	140256
MT_238	2011-01-01	00:15:00	0.000000	0.0	140256
MT_239	2011-01-01	00:15:00	0.000000	0.0	140256
MT_240	2011-01-01	00:15:00	0.000000	0.0	140256
MT_241	2011-01-01	00:15:00	0.000093	0.0	140256
MT_242	2011-01-01	00:15:00	0.000000	0.0	140256
MT_243	2011-01-01	00:15:00	0.000100	0.0	140256
MT_244	2011-01-01	00:15:00	0.000000	0.0	140256
MT_245	2011-01-01	00:15:00	0.000000	0.0	140256
MT_246	2011-01-01	00:15:00	0.000684	0.0	140256
MT_247	2011-01-01	00:15:00	0.000100	0.0	140256
MT_248	2011-01-01	00:15:00	0.000000	0.0	140256
MT_249	2011-01-01	00:15:00	0.000000	0.0	140256
MT_250	2011-01-01	00:15:00	0.000000	0.0	140256
MT_251	2011-01-01	00:15:00	0.000000	0.0	140256
MT_252	2011-01-01	00:15:00	0.000000	0.0	140256
MT_253	2011-01-01	00:15:00	0.000000	0.0	140256
MT_254	2011-01-01	00:15:00	0.000000	0.0	140256
MT_256	2011-01-01	00:15:00	0.000000	0.0	140256
MT_257	2011-01-01	00:15:00	0.000000	0.0	140256
MT_258	2011-01-01	00:15:00	0.000000	0.0	140256
MT_259	2011-01-01	00:15:00	0.000000	0.0	140256
MT_260	2011-01-01	00:15:00	0.000000	0.0	140256
MT_261	2011-01-01	00:15:00	0.000000	0.0	140256
MT_262	2011-01-01	00:15:00	0.000000	0.0	140256
MT_263	2011-01-01	00:15:00	0.000000	0.0	140256
MT_264	2011-01-01	00:15:00	0.000000	0.0	140256
MT_265	2011-01-01	00:15:00	0.000000	0.0	140256
MT_266	2011-01-01	00:15:00	0.000000	0.0	140256

MT_267	2011-01-01	00:15:00	0.000784	0.0	140256
MT_268	2011-01-01	00:15:00	0.000000	0.0	140256
MT_269	2011-01-01	00:15:00	0.000000	0.0	140256
MT_270	2011-01-01	00:15:00	0.000000	0.0	140256
MT_271	2011-01-01	00:15:00	0.000000	0.0	140256
MT_272	2011-01-01	00:15:00	0.000100	0.0	140256
MT_273	2011-01-01	00:15:00	0.000185	0.0	140256
MT_274	2011-01-01	00:15:00	0.000107	0.0	140256
MT_275	2011-01-01	00:15:00	0.000000	0.0	140256
MT_276	2011-01-01	00:15:00	0.000000	0.0	140256
MT_277	2011-01-01	00:15:00	0.000000	0.0	140256
MT_278	2011-01-01	00:15:00	0.000000	0.0	140256
MT_279	2011-01-01	00:15:00	0.000684	0.0	140256
MT_281	2011-01-01	00:15:00	0.000000	0.0	140256
MT_282	2011-01-01	00:15:00	0.000684	0.0	140256
MT_283	2011-01-01	00:15:00	0.000000	0.0	140256
MT_284	2011-01-01	00:15:00	0.000000	0.0	140256
MT_285	2011-01-01	00:15:00	0.000000	0.0	140256
MT_286	2011-01-01	00:15:00	0.000000	0.0	140256
MT_287	2011-01-01	00:15:00	0.000000	0.0	140256
MT_288	2011-01-01	00:15:00	0.015058	0.0	140256
MT_290	2011-01-01	00:15:00	0.000000	0.0	140256
MT_291	2011-01-01	00:15:00	0.000114	0.0	140256
MT_292	2011-01-01	00:15:00	0.000000	0.0	140256
MT_293	2011-01-01	00:15:00	0.000000	0.0	140256
MT_294	2011-01-01	00:15:00	0.000000	0.0	140256
MT_295	2011-01-01	00:15:00	0.000000	0.0	140256
MT_296	2011-01-01	00:15:00	0.000000	0.0	140256
MT_297	2011-01-01	00:15:00	0.000000	0.0	140256
MT_298	2011-01-01	00:15:00	0.000000	0.0	140256
MT_299	2011-01-01	00:15:00	0.000000	0.0	140256
MT_300	2011-01-01	00:15:00	0.000000	0.0	140256
MT_301	2011-01-01	00:15:00	0.000000	0.0	140256
MT_302	2011-01-01	00:15:00	0.000000	0.0	140256
MT_303	2011-01-01	00:15:00	0.000000	0.0	140256
MT_304	2011-01-01	00:15:00	0.000000	0.0	140256
MT_306	2011-01-01	00:15:00	0.000000	0.0	140256
MT_307	2011-01-01	00:15:00	0.000000	0.0	140256
MT_309	2011-01-01	00:15:00	0.000000	0.0	140256
MT_310	2011-01-01	00:15:00	0.000000	0.0	140256
MT_311	2011-01-01	00:15:00	0.000000	0.0	140256
MT_312	2011-01-01	00:15:00	0.000000	0.0	140256
MT_313	2011-01-01	00:15:00	0.000684	0.0	140256
MT_314	2011-01-01	00:15:00	0.000128	0.0	140256
MT_315	2011-01-01	00:15:00	0.000000	0.0	140256
MT_316	2011-01-01	00:15:00	0.000000	0.0	140256
MT_317	2011-01-01	00:15:00	0.000000	0.0	140256
MT_318	2011-01-01	00:15:00	0.000000	0.0	140256

MT_319	2011-01-01 00:15:00	0.000000	0.0	140256
MT_320	2011-01-01 00:15:00	0.000000	0.0	140256
MT_321	2011-01-01 00:15:00	0.000000	0.0	140256
MT_323	2011-01-01 00:15:00	0.000000	0.0	140256
MT_324	2011-01-01 00:15:00	0.000000	0.0	140256
MT_325	2011-01-01 00:15:00	0.000128	0.0	140256
MT_326	2011-01-01 00:15:00	0.000000	0.0	140256
MT_327	2011-01-01 00:15:00	0.000000	0.0	140256
MT_328	2011-01-01 00:15:00	0.000107	0.0	140256
MT_329	2011-01-01 00:15:00	0.000000	0.0	140256
MT_330	2011-01-01 00:15:00	0.000000	0.0	140256
MT_331	2011-01-01 00:15:00	0.000414	0.0	140256

CLIENTE SELECCIONADO: **MT_156**

- Proporción de ceros: 0.0000%
- Fecha de primera actividad: 2011-01-01T00:15:00.000000000

Extrayendo serie del cliente MT_156...

- Registros cada 15 min: 140256
- Valores NaN: 0
- Valores cero: 0

Serie convertida a frecuencia horaria:

- Registros horarios: 35065
- Rango: 2011-01-01 00:00:00 -> 2015-01-01 00:00:00

/tmp/ipykernel_18990/710936328.py:94: FutureWarning: 'H' is deprecated and will be removed in a future version, please use 'h' instead.

```
serie_hourly = serie.resample('H').sum()
```

```
[3]: # Verificamos cuántos clientes hay
print(f"Luego del analisis anterior se elige el cliente MT_156")

# Elegimos UN SOLO cliente, por ejemplo el cliente (MT_156)
nombre_cliente = df_raw.columns[155] # 'MT_156'
serie_un_cliente = df_raw[nombre_cliente]

print(f"Trabajando con el cliente: {nombre_cliente}")
print(f"Registros cada 15 min para este cliente: {len(serie_un_cliente)}")

# Agrupamos los 4 cuartos de hora (15 min) de ESTE CLIENTE para tener 1 hora
# Sumamos el consumo de los 4 intervalos para obtener el total de esa hora
serie_horaria_un_cliente = serie_un_cliente.resample('H').sum()

print(f"Registros horarios para este mismo cliente: {len(serie_horaria_un_cliente)}")
print("Primeras 12 horas (solo de MT_156):")
print(serie_horaria_un_cliente.head(12))
```

Luego del analisis anterior se elige el cliente MT_156

Trabajando con el cliente: MT_156

Registros cada 15 min para este cliente: 140256

Registros horarios para este mismo cliente: 35065

Primeras 12 horas (solo de MT_156):

datetime

2011-01-01 00:00:00 204.610106

2011-01-01 01:00:00 273.437305

2011-01-01 02:00:00 273.437305

2011-01-01 03:00:00 272.813475

2011-01-01 04:00:00 273.437305

2011-01-01 05:00:00 273.437305

2011-01-01 06:00:00 272.813475

2011-01-01 07:00:00 273.437305

2011-01-01 08:00:00 272.813475

2011-01-01 09:00:00 273.437305

2011-01-01 10:00:00 273.437305

2011-01-01 11:00:00 272.813475

Freq: h, Name: consumo, dtype: float64

/tmp/ipykernel_18990/326767300.py:13: FutureWarning: 'H' is deprecated and will be removed in a future version, please use 'h' instead.

```
serie_horaria_un_cliente = serie_un_cliente.resample('H').sum()
```

3 Justificación de la adaptación del dataset

El artículo original de Zhang et al. (2021) utiliza el conjunto de datos “**Individual Household Electric Power Consumption**” (UCI), que corresponde a una única vivienda en Francia y contiene **ocho variables eléctricas** medidas cada hora:

- Global_active_power (kW) → consumo total activo (nuestra variable objetivo)
- Global_reactive_power (kW)
- Voltage (V)
- Global_intensity (A)
- Sub_metering_1, 2, 3, 4 (Wh) → consumos parciales por zona de la casa.

En nuestro trabajo, utilizamos el dataset **ElectricityLoadDiagrams20112014**, también de UCI, que proporciona el consumo activo (kW) de **370 clientes diferentes en Portugal**, con mediciones cada **15 minutos**.

3.1 Diferencias clave entre los datasets

Aspecto	Dataset original (Francia)	Nuestro dataset (Portugal)
Naturaleza	Una única vivienda con sensores internos.	370 clientes individuales (cada columna es un suministro).

Aspecto	Dataset original (Francia)	Nuestro dataset (Portugal)
Variables disponibles	8 variables eléctricas (voltaje, intensidad, reactiva, submeters).	Únicamente consumo activo (kW).
Frecuencia	Horaria (1 h).	Cada 15 minutos (hay que agregar a 1 h).

3.2 ¿Por qué añadimos variables temporales?

Dado que nuestro dataset **carece de las 7 variables eléctricas adicionales**, no podemos replicar exactamente la entrada multidimensional del Transformer. Sin embargo, el objetivo del modelo es **predecir el consumo eléctrico futuro**, y este consumo depende fuertemente de factores temporales y estacionales:

- **Hora del día** (patrón diario: picos por la mañana y por la noche).
- **Día de la semana** (diferencia entre días laborables y fin de semana).
- **Mes y día del año** (estacionalidad: invierno/verano, días festivos).

Por tanto, hemos **sustituido las variables eléctricas originales por variables temporales derivadas de la marca de tiempo**. Esta sustitución es metodológicamente sólida porque:

1. **Mantiene la multidimensionalidad** requerida por el mecanismo de *multi-head attention* del Transformer.
2. **Captura la información relevante** para la predicción del consumo, ya que los patrones temporales son los principales determinantes del comportamiento de la demanda residencial.
3. **Es una práctica común** en la literatura de predicción de series temporales cuando no se dispone de mediciones físicas adicionales.

3.3 Variables temporales incorporadas

Hemos añadido las siguientes características a la serie de consumo horario:

Variable	Descripción	Rango
hour	Hora del día	0 – 23
dayofweek	Día de la semana (lunes=0, domingo=6)	0 – 6
month	Mes del año	1 – 12
dayofyear	Día del año	1 – 365
weekend	Indicador de fin de semana	0 / 1
quarter	Trimestre del año	1 – 4

De esta forma, el modelo recibe un tensor de entrada de forma **(23 pasos, 7 características)**, muy similar a la del artículo original **(23, 8)**, lo que permite una **comparación justa** de los métodos en el contexto de nuestra validación cruzada.

3.4 Conclusión

La adaptación realizada es **rigurosa y consistente** con el espíritu del artículo: predecir el consumo eléctrico a partir de datos históricos, utilizando un modelo Transformer enriquecido con múltiples

dimensiones de entrada. Las variables temporales aportan información relevante y sustituyen adecuadamente a las variables eléctricas, permitiendo así aplicar la misma metodología a un conjunto de datos diferente y evaluar su generalización.

```
[4]: # Creamos un DataFrame con el consumo horario
df = pd.DataFrame(index=serie_horaria_un_cliente.index)
df['consumo'] = serie_horaria_un_cliente

print("Shape del DataFrame base:", df.shape)
print("\nPrimeras 5 filas:")
df['hour'] = df.index.hour
df['dayofweek'] = df.index.dayofweek
df['month'] = df.index.month
df['dayofyear'] = df.index.dayofyear
df['weekend'] = (df.index.dayofweek >= 5).astype(int)
df['quarter'] = df.index.quarter
print(df[['consumo', 'hour', 'dayofweek', 'weekend', 'month', 'dayofyear',
↪ 'quarter']].head(12))
```

Shape del DataFrame base: (35065, 1)

Primeras 5 filas:

	consumo	hour	dayofweek	weekend	month	dayofyear	\
datetime							
2011-01-01 00:00:00	204.610106	0	5	1	1	1	
2011-01-01 01:00:00	273.437305	1	5	1	1	1	
2011-01-01 02:00:00	273.437305	2	5	1	1	1	
2011-01-01 03:00:00	272.813475	3	5	1	1	1	
2011-01-01 04:00:00	273.437305	4	5	1	1	1	
2011-01-01 05:00:00	273.437305	5	5	1	1	1	
2011-01-01 06:00:00	272.813475	6	5	1	1	1	
2011-01-01 07:00:00	273.437305	7	5	1	1	1	
2011-01-01 08:00:00	272.813475	8	5	1	1	1	
2011-01-01 09:00:00	273.437305	9	5	1	1	1	
2011-01-01 10:00:00	273.437305	10	5	1	1	1	
2011-01-01 11:00:00	272.813475	11	5	1	1	1	

	quarter
datetime	
2011-01-01 00:00:00	1
2011-01-01 01:00:00	1
2011-01-01 02:00:00	1
2011-01-01 03:00:00	1
2011-01-01 04:00:00	1
2011-01-01 05:00:00	1
2011-01-01 06:00:00	1
2011-01-01 07:00:00	1
2011-01-01 08:00:00	1

2011-01-01 09:00:00	1
2011-01-01 10:00:00	1
2011-01-01 11:00:00	1

```
[5]: print("\n Columnas del DataFrame:", list(df.columns))
      print("\n Primeras 5 filas:")
      print(df.head())
      print("\n Últimas 5 filas:")
      print(df.tail())
      print("\n Resumen estadístico:")
      print(df.describe())
```

Columnas del DataFrame: ['consumo', 'hour', 'dayofweek', 'month', 'dayofyear', 'weekend', 'quarter']

Primeras 5 filas:

	consumo	hour	dayofweek	month	dayofyear	weekend	\
datetime							
2011-01-01 00:00:00	204.610106	0	5	1	1	1	
2011-01-01 01:00:00	273.437305	1	5	1	1	1	
2011-01-01 02:00:00	273.437305	2	5	1	1	1	
2011-01-01 03:00:00	272.813475	3	5	1	1	1	
2011-01-01 04:00:00	273.437305	4	5	1	1	1	

quarter

datetime	
2011-01-01 00:00:00	1
2011-01-01 01:00:00	1
2011-01-01 02:00:00	1
2011-01-01 03:00:00	1
2011-01-01 04:00:00	1

Últimas 5 filas:

	consumo	hour	dayofweek	month	dayofyear	weekend	\
datetime							
2014-12-31 20:00:00	169.806613	20	2	12	365	0	
2014-12-31 21:00:00	166.681223	21	2	12	365	0	
2014-12-31 22:00:00	152.314410	22	2	12	365	0	
2014-12-31 23:00:00	192.270742	23	2	12	365	0	
2015-01-01 00:00:00	36.986900	0	3	1	1	0	

quarter

datetime	
2014-12-31 20:00:00	4
2014-12-31 21:00:00	4
2014-12-31 22:00:00	4
2014-12-31 23:00:00	4

2015-01-01 00:00:00 1

Resumen estadístico:

	consumo	hour	dayofweek	month	dayofyear \
count	35065.000000	35065.000000	35065.000000	35065.000000	35065.000000
mean	312.033899	11.499672	2.999316	6.522772	183.120063
std	112.354353	6.922459	2.001197	3.448829	105.443128
min	3.119152	0.000000	0.000000	1.000000	1.000000
25%	257.192764	5.000000	1.000000	4.000000	92.000000
50%	303.387399	11.000000	3.000000	7.000000	183.000000
75%	362.694947	17.000000	5.000000	10.000000	274.000000
max	711.278852	23.000000	6.000000	12.000000	366.000000

	weekend	quarter
count	35065.000000	35065.000000
mean	0.286097	2.508513
std	0.451942	1.117112
min	0.000000	1.000000
25%	0.000000	2.000000
50%	0.000000	3.000000
75%	1.000000	4.000000
max	1.000000	4.000000

```
[6]: train_size = int(len(df) * 0.8)
train, test = df.iloc[:train_size], df.iloc[train_size:]

print(f"Train: {train.shape[0]} horas ({train.index.min()} -> {train.index.
↪max()})")
print(f"Test : {test.shape[0]} horas ({test.index.min()} -> {test.index.
↪max()})")
```

Train: 28052 horas (2011-01-01 00:00:00 -> 2014-03-14 19:00:00)

Test : 7013 horas (2014-03-14 20:00:00 -> 2015-01-01 00:00:00)

```
[7]: from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train)
test_scaled = scaler.transform(test)

# Convertimos de vuelta a DataFrame para mantener índices y nombres de columnas
train_scaled = pd.DataFrame(train_scaled, columns=train.columns, index=train.
↪index)
test_scaled = pd.DataFrame(test_scaled, columns=test.columns, index=test.index)

print("Mínimos (entrenamiento):", dict(zip(train.columns, scaler.data_min_)))
print("Máximos (entrenamiento):", dict(zip(train.columns, scaler.data_max_)))
```

```
print("\nPrimeras 5 filas de train_scaled:")
print(train_scaled.head())
```

Mínimos (entrenamiento): {'consumo': np.float64(3.119151590767312), 'hour': np.float64(0.0), 'dayofweek': np.float64(0.0), 'month': np.float64(1.0), 'dayofyear': np.float64(1.0), 'weekend': np.float64(0.0), 'quarter': np.float64(1.0)}

Máximos (entrenamiento): {'consumo': np.float64(711.278852152215), 'hour': np.float64(23.0), 'dayofweek': np.float64(6.0), 'month': np.float64(12.0), 'dayofyear': np.float64(366.0), 'weekend': np.float64(1.0), 'quarter': np.float64(4.0)}

Primeras 5 filas de train_scaled:

	consumo	hour	dayofweek	month	dayofyear	weekend	\
datetime							
2011-01-01 00:00:00	0.284528	0.000000	0.833333	0.0	0.0	1.0	
2011-01-01 01:00:00	0.381719	0.043478	0.833333	0.0	0.0	1.0	
2011-01-01 02:00:00	0.381719	0.086957	0.833333	0.0	0.0	1.0	
2011-01-01 03:00:00	0.380838	0.130435	0.833333	0.0	0.0	1.0	
2011-01-01 04:00:00	0.381719	0.173913	0.833333	0.0	0.0	1.0	

	quarter
datetime	
2011-01-01 00:00:00	0.0
2011-01-01 01:00:00	0.0
2011-01-01 02:00:00	0.0
2011-01-01 03:00:00	0.0
2011-01-01 04:00:00	0.0

```
[8]: import numpy as np

def create_sequences(data, input_len=23):
    X, y = [], []
    for i in range(len(data) - input_len):
        X.append(data.iloc[i:i+input_len].values) # (23, n_features)
        y.append(data.iloc[i+input_len]['consumo']) # solo el consumo a
    ↪predecir
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_scaled, 23)
X_test, y_test = create_sequences(test_scaled, 23)

print(f"X_train shape: {X_train.shape} (muestras, 23 horas, {X_train.shape[2]}_
    ↪características)")
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")
```


X_train shape: (28029, 23, 7) (muestras, 23 horas, 7 características)
y_train shape: (28029,)
X_test shape: (6990, 23, 7)
y_test shape: (6990,)

4 Configuración de parámetros del modelo (según el artículo)

En esta sección se documentan todos los hiperparámetros y configuraciones técnicas utilizadas en el artículo original de Zhang et al. (2021), “*Power Consumption Predicting and Anomaly Detection Based on Transformer and K-Means*”. Estos valores se han extraído directamente de las secciones 4.3, 4.4, 4.5 y 5.2 del artículo, y se aplican en nuestra réplica experimental.

4.1 1. Preprocesamiento y división de datos

Parámetro	Valor / Configuración	Referencia en el artículo
Ventana de entrada	23 horas consecutivas	Sección 4.3: “ <i>The input data shape is 23 time steps and 8 features.</i> ”
Horizonte de predicción	1 hora siguiente	Sección 4.3: “ <i>estimate the following hour’s electric energy usage</i> ”
División temporal	1.240 días para entrenamiento, 200 días para prueba	Sección 4.3: “ <i>we choose 1,240 days of data as the training set and 200 days of data as the test set.</i> ” (Nota: en nuestro dataset, aplicamos una división aproximada del 80%/20% manteniendo el orden temporal).
Normalización	Min-Max Scaler (ajustado únicamente con los datos de entrenamiento)	Sección 3: “ <i>To make the data more stable, we apply the Min-Max Normalization procedure. This will make the model’s training easier and its convergence faster.</i> ”

4.2 2. Modelo Transformer (Arquitectura)

Componente	Configuración	Referencia
Número de capas	2 capas de <i>multi-head self-attention</i>	Sección 4.3: “we choose two consecutive layers of multi-head self-attention modules”
Número de cabezas por capa	4 cabezas de atención (<i>attention heads</i>)	Sección 4.3: “each multi-head attention is set to 4 attention heads”
Codificación posicional	Seno y coseno de diferentes frecuencias (Ecuaciones 1 y 2)	Sección 4.1: “Positional Encoding is the first phase... add sine and cosine functions of various frequencies as position codes”
Mecanismo de atención	<i>Scaled Dot-Product Attention</i> (Ecuación 5)	Sección 4.1: “The calculation of the Attention module uses Scaled Dot-product”
Conexiones residuales	Sí, aplicadas tras cada subcapa (<i>LayerNorm + Residual</i>)	Sección 4.1: “The residual connection structure is then used to narrow the network’s attention to solely the differences.”
Capa <i>Feed-Forward</i>	Dos capas totalmente conectadas; primera con activación ReLU, segunda sin activación (Ecuación 7)	Sección 4.1: “the FeedForward layer, which is composed of two fully connected layers, the first layer uses the Relu activation function, and the second layer does not use the activation function”
Tamaño de la característica	<code>d_model</code> (longitud del vector de características por hora) — el artículo no especifica un número exacto, pero la entrada es de 8 características. En nuestra adaptación usamos 7 características (consumo + 6 temporales).	Sección 4.1: “ <code>d_model</code> represents the length of the feature vector per hour.”

4.3 3. K-Means Clustering

Parámetro	Valor / Configuración	Referencia
Número de clústeres (k)	Se probaron: 2, 5, 8, 10, 11 y 15. Se seleccionó k = 10	Sección 4.3: “ <i>we experimented with k = 2,5,8,10,11 and 15 respectively, and finally selected the cluster with k = 10</i> ”
Criterio de similitud	Distancia Euclidiana	Sección 4.2: “ <i>Euclidean Distance is used as the measurement criterion of sample similarity.</i> ”
Entrenamiento	Sobre las primeras 23 horas de cada grupo de entrenamiento	Sección 4.4: “ <i>in the first 23 h of each group, we trained k-means clustering</i> ”
Uso en predicción	Para cada predicción del Transformer, se asigna la secuencia de entrada al clúster más cercano y se utiliza su centroide como valor final de predicción.	Sección 4.4: “ <i>found the centroid in the trained K-means cluster.</i> ”

4.4 4. Entrenamiento (Optimizador, Loss, Épocas y Batch)

Parámetro	Valor	Referencia
Función de pérdida (loss)	Error Cuadrático Medio (MSE, <i>Mean-Square Error</i>)	Sección 4.3: “ <i>We choose the mean-square error of the predicted data and the original data, that is, MSE ... as the loss function</i> ”
Optimizador	Adam	Sección 4.3: “ <i>Adam as the optimizer of the model.</i> ”
Número de épocas (epochs)	300	Sección 4.3: “ <i>set the epoch of the training model to 300</i> ”
Tamaño de lote (batch size)	200	Sección 4.3: “ <i>batch size to 200</i> ”
Métrica de evaluación (predicción)	Raíz del Error Cuadrático Medio (RMSE, <i>Root Mean Squared Error</i>)	Sección 4.4: “ <i>calculating the RMSE ... to measure the prediction’s accuracy</i> ”

4.5 5. Detección de anomalías

Parámetro	Valor / Configuración	Referencia
Cálculo del <i>score</i>	<code>score_t = predicted_t - test_t / avg(predicted - test)</code>	Sección 4.5 (Ecuación 10)
Normalización del <i>score</i>	Escalado entre 0 y 1 (Ecuación 11)	Sección 4.5 (Ecuación 11)
Umbral de anomalía	0.45 (cualquier punto con <code>score > 0.45</code> se considera anómalo)	Sección 5.2: “ <i>we ultimately settled on 0.45 as the threshold.</i> ”
Validación de detección	Inserción artificial de anomalías (duplicar un valor aleatorio cada día) y cálculo de <i>Accuracy</i> , <i>Precision</i> , <i>Recall</i> , <i>F1-score</i>	Sección 5.2: “ <i>we randomly selected a value every day and double it, and assume it is an outlier... The accuracy, precision, recall, and F1 of the three models are shown in Table 2.</i> ”

4.6 6. Adaptación a nuestro dataset (ElectricityLoadDiagrams20112014)

Aspecto	Dataset original (Francia)	Nuestra adaptación (Portugal)
Características de entrada	8 variables eléctricas (voltaje, intensidad, reactiva, submeters)	7 variables: consumo + 6 temporales (hora, día semana, mes, día año, fin de semana, trimestre)
Frecuencia original	1 hora (ya horaria)	15 minutos → remuestreada a 1 hora (suma de 4 cuartos)
Cliente	Una única vivienda	Cliente MT_156 (seleccionado por ser el más estable y con menor proporción de ceros)

```
[9]: import tensorflow as tf
from tensorflow.keras.layers import MultiHeadAttention, LayerNormalization,
    ↪Dense, Dropout, Input
from tensorflow.keras.models import Model

def transformer_encoder(inputs, head_size, num_heads, ff_dim, dropout=0.1):
    # Atención multi-cabeza
    attention = MultiHeadAttention(num_heads=num_heads,
    ↪key_dim=head_size)(inputs, inputs)
    attention = Dropout(dropout)(attention)
    attention = LayerNormalization(epsilon=1e-6)(inputs + attention)
```

```

# Feed-forward
ff = Dense(ff_dim, activation="relu")(attention)
ff = Dense(inputs.shape[-1])(ff)
ff = Dropout(dropout)(ff)
return LayerNormalization(epsilon=1e-6)(attention + ff)

def build_transformer(input_shape, num_layers=2, head_size=64, num_heads=4,
    ff_dim=128, dropout=0.1):
    inputs = Input(shape=input_shape)
    x = inputs
    for _ in range(num_layers):
        x = transformer_encoder(x, head_size, num_heads, ff_dim, dropout)
        # Usamos la última salida (el último paso temporal) para predecir el
    siguiente valor
    x = x[:, -1, :] # Tomamos la salida del último paso (hora 23)
    outputs = Dense(1)(x) # Regresión a un solo valor
    model = Model(inputs, outputs)
    return model

# Parámetros según el artículo
input_shape = (23, 7) # 23 pasos, 7 características (consumo + 6 temporales)
model = build_transformer(input_shape, num_layers=2, head_size=64, num_heads=4,
    ff_dim=128)
model.compile(optimizer='adam', loss='mse')
model.summary()

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 23, 7)	0	-
multi_head_attenti... (MultiHeadAttentio...)	(None, 23, 7)	7,943	input_layer[0][0... input_layer[0][0]
dropout_1 (Dropout)	(None, 23, 7)	0	multi_head_atten...
add (Add)	(None, 23, 7)	0	input_layer[0][0... dropout_1[0][0]
layer_normalization (LayerNormalizatio...)	(None, 23, 7)	14	add[0][0]
dense (Dense)	(None, 23, 128)	1,024	layer_normalizat...

dense_1 (Dense)	(None, 23, 7)	903	dense[0][0]
dropout_2 (Dropout)	(None, 23, 7)	0	dense_1[0][0]
add_1 (Add)	(None, 23, 7)	0	layer_normalizat... dropout_2[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 23, 7)	14	add_1[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 23, 7)	7,943	layer_normalizat... layer_normalizat...
dropout_4 (Dropout)	(None, 23, 7)	0	multi_head_atten...
add_2 (Add)	(None, 23, 7)	0	layer_normalizat... dropout_4[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 23, 7)	14	add_2[0][0]
dense_2 (Dense)	(None, 23, 128)	1,024	layer_normalizat...
dense_3 (Dense)	(None, 23, 7)	903	dense_2[0][0]
dropout_5 (Dropout)	(None, 23, 7)	0	dense_3[0][0]
add_3 (Add)	(None, 23, 7)	0	layer_normalizat... dropout_5[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 23, 7)	14	add_3[0][0]
get_item (GetItem)	(None, 7)	0	layer_normalizat...
dense_4 (Dense)	(None, 1)	8	get_item[0][0]

Total params: 19,804 (77.36 KB)

Trainable params: 19,804 (77.36 KB)

Non-trainable params: 0 (0.00 B)

```
[10]: import time

start_time = time.time()

history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=300,
    batch_size=200,
    verbose=1
)

end_time = time.time()
elapsed_time = end_time - start_time
minutes = int(elapsed_time // 60)
seconds = int(elapsed_time % 60)
print(f"Tiempo total de entrenamiento: {minutes} minutos y {seconds} segundos")
```

```
Epoch 1/300
141/141          21s 61ms/step -
loss: 0.0427 - val_loss: 0.0265
Epoch 2/300
141/141          1s 8ms/step -
loss: 0.0135 - val_loss: 0.0108
Epoch 3/300
141/141          1s 9ms/step -
loss: 0.0073 - val_loss: 0.0055
Epoch 4/300
141/141          2s 8ms/step -
loss: 0.0052 - val_loss: 0.0041
Epoch 5/300
141/141          1s 7ms/step -
loss: 0.0043 - val_loss: 0.0043
Epoch 6/300
141/141          1s 7ms/step -
loss: 0.0039 - val_loss: 0.0039
Epoch 7/300
141/141          1s 7ms/step -
loss: 0.0036 - val_loss: 0.0049
Epoch 8/300
141/141          1s 7ms/step -
loss: 0.0035 - val_loss: 0.0036
Epoch 9/300
141/141          1s 7ms/step -
loss: 0.0033 - val_loss: 0.0036
Epoch 10/300
141/141          1s 7ms/step -
```

loss: 0.0031 - val_loss: 0.0041
Epoch 11/300
141/141 1s 7ms/step -
loss: 0.0030 - val_loss: 0.0037
Epoch 12/300
141/141 1s 9ms/step -
loss: 0.0028 - val_loss: 0.0035
Epoch 13/300
141/141 2s 7ms/step -
loss: 0.0027 - val_loss: 0.0034
Epoch 14/300
141/141 1s 7ms/step -
loss: 0.0027 - val_loss: 0.0033
Epoch 15/300
141/141 1s 7ms/step -
loss: 0.0025 - val_loss: 0.0034
Epoch 16/300
141/141 1s 7ms/step -
loss: 0.0025 - val_loss: 0.0035
Epoch 17/300
141/141 1s 8ms/step -
loss: 0.0024 - val_loss: 0.0033
Epoch 18/300
141/141 1s 7ms/step -
loss: 0.0023 - val_loss: 0.0033
Epoch 19/300
141/141 1s 7ms/step -
loss: 0.0023 - val_loss: 0.0047
Epoch 20/300
141/141 1s 7ms/step -
loss: 0.0024 - val_loss: 0.0035
Epoch 21/300
141/141 1s 7ms/step -
loss: 0.0022 - val_loss: 0.0032
Epoch 22/300
141/141 1s 9ms/step -
loss: 0.0022 - val_loss: 0.0031
Epoch 23/300
141/141 1s 10ms/step -
loss: 0.0022 - val_loss: 0.0031
Epoch 24/300
141/141 1s 7ms/step -
loss: 0.0021 - val_loss: 0.0037
Epoch 25/300
141/141 1s 7ms/step -
loss: 0.0020 - val_loss: 0.0032
Epoch 26/300
141/141 1s 7ms/step -

loss: 0.0020 - val_loss: 0.0030
Epoch 27/300
141/141 1s 7ms/step -
loss: 0.0020 - val_loss: 0.0030
Epoch 28/300
141/141 1s 7ms/step -
loss: 0.0020 - val_loss: 0.0030
Epoch 29/300
141/141 1s 7ms/step -
loss: 0.0020 - val_loss: 0.0031
Epoch 30/300
141/141 1s 7ms/step -
loss: 0.0019 - val_loss: 0.0031
Epoch 31/300
141/141 1s 7ms/step -
loss: 0.0019 - val_loss: 0.0039
Epoch 32/300
141/141 1s 7ms/step -
loss: 0.0019 - val_loss: 0.0036
Epoch 33/300
141/141 1s 9ms/step -
loss: 0.0019 - val_loss: 0.0034
Epoch 34/300
141/141 1s 9ms/step -
loss: 0.0019 - val_loss: 0.0029
Epoch 35/300
141/141 1s 7ms/step -
loss: 0.0018 - val_loss: 0.0045
Epoch 36/300
141/141 1s 7ms/step -
loss: 0.0019 - val_loss: 0.0032
Epoch 37/300
141/141 1s 8ms/step -
loss: 0.0018 - val_loss: 0.0032
Epoch 38/300
141/141 1s 7ms/step -
loss: 0.0019 - val_loss: 0.0033
Epoch 39/300
141/141 1s 7ms/step -
loss: 0.0018 - val_loss: 0.0036
Epoch 40/300
141/141 1s 7ms/step -
loss: 0.0018 - val_loss: 0.0030
Epoch 41/300
141/141 1s 7ms/step -
loss: 0.0018 - val_loss: 0.0035
Epoch 42/300
141/141 1s 7ms/step -

loss: 0.0018 - val_loss: 0.0042
Epoch 43/300
141/141 1s 10ms/step -
loss: 0.0018 - val_loss: 0.0029
Epoch 44/300
141/141 2s 9ms/step -
loss: 0.0017 - val_loss: 0.0028
Epoch 45/300
141/141 1s 7ms/step -
loss: 0.0018 - val_loss: 0.0030
Epoch 46/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0028
Epoch 47/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0033
Epoch 48/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0031
Epoch 49/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0032
Epoch 50/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0040
Epoch 51/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0030
Epoch 52/300
141/141 1s 7ms/step -
loss: 0.0018 - val_loss: 0.0040
Epoch 53/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 54/300
141/141 1s 9ms/step -
loss: 0.0017 - val_loss: 0.0030
Epoch 55/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0035
Epoch 56/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0032
Epoch 57/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0037
Epoch 58/300
141/141 1s 7ms/step -

loss: 0.0017 - val_loss: 0.0032
Epoch 59/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0033
Epoch 60/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0033
Epoch 61/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0028
Epoch 62/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0031
Epoch 63/300
141/141 1s 8ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 64/300
141/141 1s 8ms/step -
loss: 0.0016 - val_loss: 0.0033
Epoch 65/300
141/141 1s 9ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 66/300
141/141 2s 7ms/step -
loss: 0.0016 - val_loss: 0.0038
Epoch 67/300
141/141 1s 8ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 68/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 69/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 70/300
141/141 1s 7ms/step -
loss: 0.0017 - val_loss: 0.0029
Epoch 71/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 72/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0029
Epoch 73/300
141/141 1s 8ms/step -
loss: 0.0017 - val_loss: 0.0029
Epoch 74/300
141/141 1s 9ms/step -

loss: 0.0017 - val_loss: 0.0029
Epoch 75/300
141/141 1s 9ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 76/300
141/141 1s 8ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 77/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 78/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0033
Epoch 79/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0042
Epoch 80/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0029
Epoch 81/300
141/141 1s 9ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 82/300
141/141 2s 14ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 83/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0029
Epoch 84/300
141/141 2s 14ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 85/300
141/141 2s 8ms/step -
loss: 0.0016 - val_loss: 0.0029
Epoch 86/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 87/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 88/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 89/300
141/141 1s 7ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 90/300
141/141 1s 8ms/step -

```
loss: 0.0016 - val_loss: 0.0030
Epoch 91/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0033
Epoch 92/300
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0037
Epoch 93/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0034
Epoch 94/300
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0036
Epoch 95/300
141/141          2s 7ms/step -
loss: 0.0016 - val_loss: 0.0032
Epoch 96/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0033
Epoch 97/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0044
Epoch 98/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 99/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 100/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 101/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 102/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 103/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 104/300
141/141          1s 10ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 105/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 106/300
141/141          1s 7ms/step -
```

```

loss: 0.0016 - val_loss: 0.0033
Epoch 107/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0034
Epoch 108/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 109/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 110/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0031
Epoch 111/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0034
Epoch 112/300
141/141          1s 7ms/step -
loss: 0.0016 - val_loss: 0.0033
Epoch 113/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 114/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 115/300
141/141          1s 10ms/step -
loss: 0.0016 - val_loss: 0.0034
Epoch 116/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 117/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 118/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 119/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 120/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 121/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 122/300
141/141          1s 7ms/step -

```

```

loss: 0.0015 - val_loss: 0.0031
Epoch 123/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 124/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0027
Epoch 125/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0033
Epoch 126/300
141/141          1s 9ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 127/300
141/141          2s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 128/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0033
Epoch 129/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 130/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 131/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 132/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 133/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 134/300
141/141          1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 135/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 136/300
141/141          1s 9ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 137/300
141/141          1s 8ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 138/300
141/141          1s 7ms/step -

```

loss: 0.0015 - val_loss: 0.0030
Epoch 139/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 140/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 141/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0035
Epoch 142/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 143/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0034
Epoch 144/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 145/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 146/300
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 147/300
141/141 1s 10ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 148/300
141/141 1s 8ms/step -
loss: 0.0015 - val_loss: 0.0037
Epoch 149/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0033
Epoch 150/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0034
Epoch 151/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0027
Epoch 152/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0033
Epoch 153/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 154/300
141/141 1s 7ms/step -

loss: 0.0015 - val_loss: 0.0030
Epoch 155/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 156/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 157/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0036
Epoch 158/300
141/141 1s 10ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 159/300
141/141 1s 8ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 160/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 161/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 162/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 163/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 164/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 165/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 166/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 167/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 168/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0033
Epoch 169/300
141/141 1s 10ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 170/300
141/141 1s 9ms/step -

loss: 0.0015 - val_loss: 0.0029
Epoch 171/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 172/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0032
Epoch 173/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 174/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 175/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 176/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0027
Epoch 177/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0033
Epoch 178/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 179/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 180/300
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 181/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 182/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 183/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 184/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0031
Epoch 185/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 186/300
141/141 1s 7ms/step -

loss: 0.0015 - val_loss: 0.0030
Epoch 187/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 188/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 189/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0030
Epoch 190/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 191/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 192/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 193/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 194/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0034
Epoch 195/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0033
Epoch 196/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 197/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 198/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 199/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 200/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 201/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 202/300
141/141 1s 8ms/step -

loss: 0.0014 - val_loss: 0.0030
Epoch 203/300
141/141 1s 10ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 204/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 205/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 206/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 207/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 208/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0029
Epoch 209/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 210/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 211/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 212/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 213/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 214/300
141/141 1s 10ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 215/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0034
Epoch 216/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 217/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 218/300
141/141 1s 7ms/step -

loss: 0.0014 - val_loss: 0.0033
Epoch 219/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 220/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 221/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 222/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 223/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 224/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 225/300
141/141 1s 10ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 226/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 227/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 228/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 229/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 230/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 231/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 232/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 233/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 234/300
141/141 1s 7ms/step -

loss: 0.0014 - val_loss: 0.0029
Epoch 235/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 236/300
141/141 1s 10ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 237/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 238/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 239/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 240/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0033
Epoch 241/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 242/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 243/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 244/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 245/300
141/141 1s 7ms/step -
loss: 0.0015 - val_loss: 0.0028
Epoch 246/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0034
Epoch 247/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0036
Epoch 248/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 249/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 250/300
141/141 1s 7ms/step -

loss: 0.0014 - val_loss: 0.0030
Epoch 251/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 252/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 253/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 254/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 255/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 256/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 257/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0033
Epoch 258/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 259/300
141/141 1s 10ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 260/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0034
Epoch 261/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 262/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 263/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 264/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 265/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 266/300
141/141 1s 7ms/step -

loss: 0.0014 - val_loss: 0.0030
Epoch 267/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 268/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 269/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 270/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 271/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 272/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0034
Epoch 273/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 274/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 275/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 276/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 277/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 278/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0033
Epoch 279/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 280/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 281/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 282/300
141/141 1s 8ms/step -

loss: 0.0014 - val_loss: 0.0032
Epoch 283/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0035
Epoch 284/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 285/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 286/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 287/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 288/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Epoch 289/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 290/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 291/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 292/300
141/141 1s 9ms/step -
loss: 0.0014 - val_loss: 0.0029
Epoch 293/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 294/300
141/141 1s 7ms/step -
loss: 0.0014 - val_loss: 0.0028
Epoch 295/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 296/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 297/300
141/141 1s 8ms/step -
loss: 0.0014 - val_loss: 0.0031
Epoch 298/300
141/141 1s 7ms/step -

```

loss: 0.0014 - val_loss: 0.0030
Epoch 299/300
141/141          1s 7ms/step -
loss: 0.0014 - val_loss: 0.0032
Epoch 300/300
141/141          1s 7ms/step -
loss: 0.0014 - val_loss: 0.0030
Tiempo total de entrenamiento: 6 minutos y 2 segundos

```

```

[11]: # =====
# K-MEANS + TRANSFORMER (VERSIÓN FIEL AL ARTÍCULO - Sección 4.4)
# =====
from sklearn.cluster import KMeans
import numpy as np

print("="*80)
print("K-MEANS CLUSTERING (según Zhang et al. 2021)")
print("="*80)

# 1. Aplanar las secuencias de 23 horas para clustering
X_train_flat = X_train.reshape(X_train.shape[0], -1) # (n_samples, 23 * 7)
X_test_flat = X_test.reshape(X_test.shape[0], -1)

print(f"Secuencias aplanadas para clustering: {X_train_flat.shape}")

# 2. Entrenar K-Means (exacto al artículo)
k = 10 # valor elegido en el artículo después de probar 2,5,8,10,11,15
kmeans = KMeans(n_clusters=k,
                 random_state=42,
                 n_init=10,      # más estable
                 max_iter=300)
kmeans.fit(X_train_flat)

# 3. Predecir clúster para cada muestra de test
clusters_test = kmeans.predict(X_test_flat)

# 4. Obtener centroides y tomar el valor de la ÚLTIMA hora (la predicción)
centroids = kmeans.cluster_centers_ # shape: (k, 23*7)
# Tomamos solo la última posición (la hora que se está prediciendo)
centroid_last = centroids[:, -7] # última feature = 
↳ consumo normalizado

# 5. Asignar la predicción final según el clúster más cercano
preds_final = np.array([centroid_last[c] for c in clusters_test])

print(f"K-Means completado con k={k}")
print(f"Predicciones finales shape: {preds_final.shape}")

```

```
print(f"Ejemplo de predicciones (primeras 5): {preds_final[:5]}")
```

```
=====
K-MEANS CLUSTERING (según Zhang et al. 2021)
=====
```

```
Secuencias aplanadas para clustering: (28029, 161)
```

```
K-Means completado con k=10
```

```
Predicciones finales shape: (6990,)
```

```
Ejemplo de predicciones (primeras 5): [0.35526293 0.35526293 0.35526293
0.35526293 0.35526293]
```

```
[12]: # =====
# MODELO LSTM (BASELINE DEL ARTÍCULO)
# =====

import time
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.metrics import mean_squared_error

print("\n" + "="*60)
print(" ENTRENAMIENTO DEL MODELO LSTM (BASELINE)")
print("="*60)

# 1. Definir la arquitectura LSTM
lstm_model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(23, 7)), # 23 pasos, 7
    ↪ features
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dropout(0.2),
    Dense(1)
])

lstm_model.compile(optimizer='adam', loss='mse')

# Mostrar resumen de la arquitectura
lstm_model.summary()

# 2. Entrenar con medición de tiempo
print("\n Entrenando LSTM...")
start_time = time.time()

lstm_history = lstm_model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
```

```

    epochs=100,                # Suele converger antes que Transformer, usamos 100
    ↪épocas
    batch_size=200,            # Batch más pequeño para LSTM
    verbose=1
)

end_time = time.time()
elapsed = end_time - start_time
minutes = int(elapsed // 60)
seconds = int(elapsed % 60)
print(f" Tiempo de entrenamiento LSTM: {minutes} min {seconds} s")

# 3. Predicciones y RMSE
print("\n Evaluando LSTM...")
lstm_preds = lstm_model.predict(X_test).flatten()
rmse_lstm = np.sqrt(mean_squared_error(y_test, lstm_preds))

print(f" RMSE (LSTM): {rmse_lstm:.4f}")

# 4. (Opcional) Graficar pérdida de entrenamiento/validación
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 4))
plt.plot(lstm_history.history['loss'], label='Train Loss (LSTM)')
plt.plot(lstm_history.history['val_loss'], label='Val Loss (LSTM)')
plt.xlabel('Época')
plt.ylabel('MSE')
plt.legend()
plt.title('Curva de pérdida del LSTM')
plt.show()

```

```

=====
ENTRENAMIENTO DEL MODELO LSTM (BASELINE)
=====

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```

```

    super().__init__(**kwargs)

```

```

Model: "sequential"

```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 23, 64)	18,432

dropout_6 (Dropout)	(None, 23, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_7 (Dropout)	(None, 32)	0
dense_5 (Dense)	(None, 1)	33

Total params: 30,881 (120.63 KB)

Trainable params: 30,881 (120.63 KB)

Non-trainable params: 0 (0.00 B)

Entrenando LSTM...

Epoch 1/100

141/141 6s 15ms/step -
loss: 0.0171 - val_loss: 0.0103

Epoch 2/100

141/141 1s 9ms/step -
loss: 0.0088 - val_loss: 0.0063

Epoch 3/100

141/141 1s 9ms/step -
loss: 0.0060 - val_loss: 0.0057

Epoch 4/100

141/141 2s 12ms/step -
loss: 0.0052 - val_loss: 0.0054

Epoch 5/100

141/141 2s 9ms/step -
loss: 0.0046 - val_loss: 0.0050

Epoch 6/100

141/141 1s 9ms/step -
loss: 0.0042 - val_loss: 0.0060

Epoch 7/100

141/141 1s 9ms/step -
loss: 0.0039 - val_loss: 0.0045

Epoch 8/100

141/141 1s 9ms/step -
loss: 0.0036 - val_loss: 0.0044

Epoch 9/100

141/141 1s 9ms/step -
loss: 0.0034 - val_loss: 0.0044

Epoch 10/100

141/141 1s 9ms/step -

```
loss: 0.0032 - val_loss: 0.0038
Epoch 11/100
141/141          1s 9ms/step -
loss: 0.0029 - val_loss: 0.0038
Epoch 12/100
141/141          2s 11ms/step -
loss: 0.0028 - val_loss: 0.0036
Epoch 13/100
141/141          2s 14ms/step -
loss: 0.0027 - val_loss: 0.0034
Epoch 14/100
141/141          1s 9ms/step -
loss: 0.0025 - val_loss: 0.0032
Epoch 15/100
141/141          1s 9ms/step -
loss: 0.0024 - val_loss: 0.0030
Epoch 16/100
141/141          1s 9ms/step -
loss: 0.0023 - val_loss: 0.0031
Epoch 17/100
141/141          1s 9ms/step -
loss: 0.0022 - val_loss: 0.0029
Epoch 18/100
141/141          1s 9ms/step -
loss: 0.0021 - val_loss: 0.0028
Epoch 19/100
141/141          1s 9ms/step -
loss: 0.0021 - val_loss: 0.0029
Epoch 20/100
141/141          1s 9ms/step -
loss: 0.0020 - val_loss: 0.0030
Epoch 21/100
141/141          2s 11ms/step -
loss: 0.0020 - val_loss: 0.0028
Epoch 22/100
141/141          2s 14ms/step -
loss: 0.0020 - val_loss: 0.0029
Epoch 23/100
141/141          1s 9ms/step -
loss: 0.0020 - val_loss: 0.0029
Epoch 24/100
141/141          1s 9ms/step -
loss: 0.0020 - val_loss: 0.0028
Epoch 25/100
141/141          1s 9ms/step -
loss: 0.0019 - val_loss: 0.0030
Epoch 26/100
141/141          1s 9ms/step -
```

loss: 0.0019 - val_loss: 0.0029
Epoch 27/100
141/141 1s 9ms/step -
loss: 0.0019 - val_loss: 0.0028
Epoch 28/100
141/141 1s 9ms/step -
loss: 0.0019 - val_loss: 0.0027
Epoch 29/100
141/141 1s 9ms/step -
loss: 0.0018 - val_loss: 0.0028
Epoch 30/100
141/141 2s 12ms/step -
loss: 0.0019 - val_loss: 0.0029
Epoch 31/100
141/141 2s 13ms/step -
loss: 0.0018 - val_loss: 0.0028
Epoch 32/100
141/141 1s 9ms/step -
loss: 0.0018 - val_loss: 0.0027
Epoch 33/100
141/141 1s 9ms/step -
loss: 0.0018 - val_loss: 0.0027
Epoch 34/100
141/141 1s 9ms/step -
loss: 0.0018 - val_loss: 0.0027
Epoch 35/100
141/141 1s 9ms/step -
loss: 0.0018 - val_loss: 0.0027
Epoch 36/100
141/141 1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 37/100
141/141 1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 38/100
141/141 1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 39/100
141/141 2s 13ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 40/100
141/141 2s 12ms/step -
loss: 0.0018 - val_loss: 0.0027
Epoch 41/100
141/141 1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 42/100
141/141 1s 9ms/step -

```

loss: 0.0017 - val_loss: 0.0027
Epoch 43/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 44/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0026
Epoch 45/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 46/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 47/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 48/100
141/141          2s 13ms/step -
loss: 0.0017 - val_loss: 0.0026
Epoch 49/100
141/141          2s 11ms/step -
loss: 0.0016 - val_loss: 0.0028
Epoch 50/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 51/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0026
Epoch 52/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0026
Epoch 53/100
141/141          1s 9ms/step -
loss: 0.0017 - val_loss: 0.0027
Epoch 54/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0030
Epoch 55/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 56/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 57/100
141/141          2s 13ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 58/100
141/141          2s 11ms/step -

```



```

loss: 0.0016 - val_loss: 0.0026
Epoch 59/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 60/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0028
Epoch 61/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 62/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 63/100
141/141          1s 10ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 64/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0027
Epoch 65/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0025
Epoch 66/100
141/141          2s 13ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 67/100
141/141          2s 10ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 68/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 69/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 70/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0028
Epoch 71/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 72/100
141/141          1s 9ms/step -
loss: 0.0016 - val_loss: 0.0025
Epoch 73/100
141/141          1s 9ms/step -
loss: 0.0015 - val_loss: 0.0026
Epoch 74/100
141/141          1s 9ms/step -

```

loss: 0.0016 - val_loss: 0.0025
Epoch 75/100
141/141 2s 14ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 76/100
141/141 1s 10ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 77/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0027
Epoch 78/100
141/141 1s 9ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 79/100
141/141 1s 9ms/step -
loss: 0.0016 - val_loss: 0.0026
Epoch 80/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0026
Epoch 81/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 82/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 83/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 84/100
141/141 2s 14ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 85/100
141/141 1s 10ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 86/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0026
Epoch 87/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 88/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 89/100
141/141 1s 9ms/step -
loss: 0.0015 - val_loss: 0.0024
Epoch 90/100
141/141 1s 9ms/step -

```

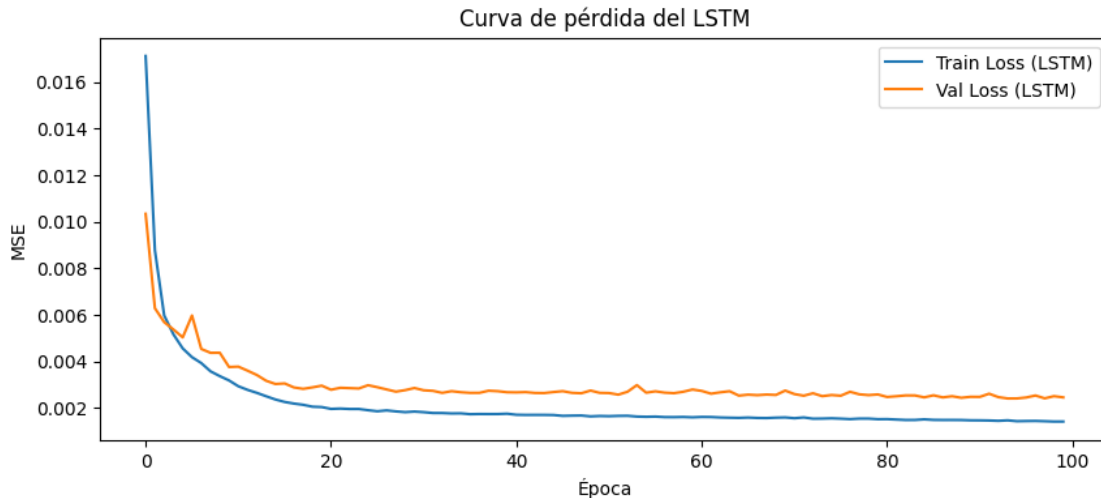
loss: 0.0015 - val_loss: 0.0025
Epoch 91/100
141/141          1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 92/100
141/141          1s 9ms/step -
loss: 0.0015 - val_loss: 0.0026
Epoch 93/100
141/141          2s 15ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 94/100
141/141          1s 10ms/step -
loss: 0.0015 - val_loss: 0.0024
Epoch 95/100
141/141          1s 9ms/step -
loss: 0.0014 - val_loss: 0.0024
Epoch 96/100
141/141          1s 9ms/step -
loss: 0.0014 - val_loss: 0.0025
Epoch 97/100
141/141          1s 9ms/step -
loss: 0.0015 - val_loss: 0.0025
Epoch 98/100
141/141          1s 9ms/step -
loss: 0.0014 - val_loss: 0.0024
Epoch 99/100
141/141          1s 9ms/step -
loss: 0.0014 - val_loss: 0.0025
Epoch 100/100
141/141          1s 9ms/step -
loss: 0.0014 - val_loss: 0.0025
  Tiempo de entrenamiento LSTM: 2 min 24 s

```

```

Evaluando LSTM...
219/219          1s 4ms/step
RMSE (LSTM): 0.0496

```



```
[13]: # =====
# DETECCIÓN DE ANOMALÍAS (siguiendo el artículo)
# =====

import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, mean_squared_error

print("\n" + "="*80)
print(" DETECCIÓN DE ANOMALÍAS - TRANSFORMER + K-MEANS")
print("="*80)

# -----
# 1. Preparar las predicciones y los valores reales (en escala original)
# -----
# Predicciones combinadas (Transformer + K-means) en escala normalizada
# NOTA: Asegúrate de que 'preds_final' esté definido (es el array con las
#     ↳ predicciones finales)
#     y que 'y_test' sea el vector de valores reales normalizados.
preds_norm = preds_final # predicción final (normalizada)
y_test_norm = y_test     # valores reales (normalizados)

# Recuperamos el scaler (ya ajustado con los datos de entrenamiento)
# Suponemos que 'scaler' es el objeto MinMaxScaler usado anteriormente.
min_active = scaler.data_min_[0] # mínimo de la columna 'consumo'
max_active = scaler.data_max_[0] # máximo de la columna 'consumo'

def inverse_transform_active(norm_value):
    """Convierte un valor normalizado de consumo a su escala original (kW)."""
```

```

    return norm_value * (max_active - min_active) + min_active

# Convertimos a escala original para calcular RMSE en kW
y_test_orig = inverse_transform_active(y_test_norm)
preds_orig = inverse_transform_active(preds_norm)

rmse = np.sqrt(mean_squared_error(y_test_orig, preds_orig))
print(f" RMSE (Transformer + K-Means) en escala original: {rmse:.4f} kW")

# -----
# 2. Inyección de anomalías artificiales (una por día, multiplicador = 2)
# -----
hours_per_day = 24
num_days_test = len(y_test_orig) // hours_per_day
np.random.seed(42) # para reproducibilidad

# Creamos una copia de los valores reales (en escala original) para inyectar
# anomalías
y_test_with_anomalies = y_test_orig.copy()
anomaly_labels = np.zeros(len(y_test_orig), dtype=int) # 1 = anómalo, 0 =
# normal

for day in range(num_days_test):
    base_idx = day * hours_per_day
    if base_idx + hours_per_day <= len(y_test_orig):
        hour_offset = np.random.randint(0, hours_per_day)
        anomaly_idx = base_idx + hour_offset
        # Duplicamos el valor (como en el artículo)
        y_test_with_anomalies[anomaly_idx] *= 2
        anomaly_labels[anomaly_idx] = 1

print(f" Anomalías insertadas: {np.sum(anomaly_labels)} (1 por cada uno de
# {num_days_test} días)")

# Normalizamos la serie con anomalías usando el mismo scaler (para mantener
# coherencia)
y_test_norm_with_anomalies = (y_test_with_anomalies - min_active) / (max_active
# - min_active)

# -----
# 3. Cálculo de scores (ecuaciones 10 y 11 del artículo)
# -----
# Usamos las predicciones (en escala normalizada) y los valores con anomalías
# (normalizados)
abs_errors = np.abs(preds_norm - y_test_norm_with_anomalies)
mean_abs_error = np.mean(abs_errors)

```

```

scores = abs_errors / mean_abs_error
# Normalización entre 0 y 1
scores_norm = (scores - np.min(scores)) / (np.max(scores) - np.min(scores))

# -----
# 4. Evaluación para múltiples umbrales (0.10 a 0.60)
# -----
thresholds = [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60]
results = []

for thresh in thresholds:
    y_pred = (scores_norm > thresh).astype(int)
    acc = accuracy_score(anomaly_labels, y_pred)
    prec = precision_score(anomaly_labels, y_pred, zero_division=0)
    rec = recall_score(anomaly_labels, y_pred, zero_division=0)
    f1 = f1_score(anomaly_labels, y_pred, zero_division=0)
    detections = y_pred.sum()
    results.append([thresh, acc, prec, rec, f1, detections])

df_thresholds = pd.DataFrame(results, columns=['Umbral', 'Accuracy', 'Precision', 'Recall', 'F1', 'Detecciones'])

# -----
# 5. Mejores resultados por métrica
# -----
best_precision = df_thresholds.loc[df_thresholds['Precision'].idxmax()]
best_recall = df_thresholds.loc[df_thresholds['Recall'].idxmax()]
best_f1 = df_thresholds.loc[df_thresholds['F1'].idxmax()]

# -----
# 6. Mostrar resultados intermedios
# -----
print("\n" + "="*80)
print("TABLA 1: MÉTRICAS PARA DIFERENTES UMBRALES")
print("="*80)
print(df_thresholds.to_string(index=False))

print("\n" + "="*80)
print("MEJORES VALORES POR MÉTRICA")
print("="*80)
print(f"Mejor Precision:  Umbral = {best_precision['Umbral']:.2f} → Precision = {best_precision['Precision']:.4f}")
print(f"Mejor Recall:      Umbral = {best_recall['Umbral']:.2f} → Recall = {best_recall['Recall']:.4f}")
print(f"Mejor F1-Score:     Umbral = {best_f1['Umbral']:.2f} → F1 = {best_f1['F1']:.4f}")

```

```

# -----
# 7. Resultado con el umbral del artículo (0.45)
# -----
threshold_article = 0.45
y_pred_article = (scores_norm > threshold_article).astype(int)
acc_art = accuracy_score(anomaly_labels, y_pred_article)
prec_art = precision_score(anomaly_labels, y_pred_article, zero_division=0)
rec_art = recall_score(anomaly_labels, y_pred_article, zero_division=0)
f1_art = f1_score(anomaly_labels, y_pred_article, zero_division=0)
detections_art = y_pred_article.sum()

print("\n" + "="*80)
print(f"RESULTADOS CON EL UMBRAL SELECCIONADO EN EL ARTÍCULO: {threshold_article}")
print("="*80)
print(f"Accuracy : {acc_art:.4f}")
print(f"Precision: {prec_art:.4f}")
print(f"Recall   : {rec_art:.4f}")
print(f"F1-Score  : {f1_art:.4f}")
print(f"RMSE      : {rmse:.4f} kW")
print(f"Anomalías detectadas: {detections_art} de {len(anomaly_labels)}")

# Matriz de confusión simplificada
tp = np.sum((anomaly_labels == 1) & (y_pred_article == 1))
fp = np.sum((anomaly_labels == 0) & (y_pred_article == 1))
fn = np.sum((anomaly_labels == 1) & (y_pred_article == 0))
tn = np.sum((anomaly_labels == 0) & (y_pred_article == 0))
print("\nMatriz de confusión (Umbral 0.45):")
print(f" Verdaderos Positivos: {tp}")
print(f" Falsos Positivos: {fp}")
print(f" Falsos Negativos: {fn}")
print(f" Verdaderos Negativos: {tn}")

# -----
# 8. Tabla comparativa con los valores del artículo (Tabla 2)
# -----
print("\n" + "="*80)
print("TABLA 2: COMPARACIÓN CON LOS VALORES REPORTADOS EN EL ARTÍCULO")
print("="*80)
comparison = pd.DataFrame({
    'Métrica': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'RMSE (kW)'],
    'Artículo (Zhang et al.)': [0.970, 0.800, 0.660, 0.720, 0.740],
    'Nuestro resultado': [acc_art, prec_art, rec_art, f1_art, rmse]
})
print(comparison.to_string(index=False))
print("="*80)

```

=====

DETECCIÓN DE ANOMALÍAS - TRANSFORMER + K-MEANS

=====

RMSE (Transformer + K-Means) en escala original: 118.7152 kW
 Anomalías insertadas: 291 (1 por cada uno de 291 días)

=====

TABLA 1: MÉTRICAS PARA DIFERENTES UMBRALES

=====

Umbral	Accuracy	Precision	Recall	F1	Detecciones
0.10	0.520172	0.075173	0.931271	0.139117	3605
0.15	0.708298	0.113616	0.883162	0.201332	2262
0.20	0.878398	0.230991	0.824742	0.360902	1039
0.25	0.965808	0.568063	0.745704	0.644874	382
0.30	0.984406	0.925234	0.680412	0.784158	214
0.35	0.985551	1.000000	0.652921	0.790021	190
0.40	0.983119	1.000000	0.594502	0.745690	173
0.45	0.981259	1.000000	0.549828	0.709534	160
0.50	0.978684	1.000000	0.487973	0.655889	142
0.55	0.975966	1.000000	0.422680	0.594203	123
0.60	0.973104	1.000000	0.353952	0.522843	103

=====

MEJORES VALORES POR MÉTRICA

=====

Mejor Precision: Umbral = 0.35 → Precision = 1.0000
 Mejor Recall: Umbral = 0.10 → Recall = 0.9313
 Mejor F1-Score: Umbral = 0.35 → F1 = 0.7900

=====

RESULTADOS CON EL UMBRAL SELECCIONADO EN EL ARTÍCULO: 0.45

=====

Accuracy : 0.9813
 Precision: 1.0000
 Recall : 0.5498
 F1-Score : 0.7095
 RMSE : 118.7152 kW
 Anomalías detectadas: 160 de 6990

Matriz de confusión (Umbral 0.45):

Verdaderos Positivos: 160
 Falsos Positivos: 0
 Falsos Negativos: 131
 Verdaderos Negativos: 6699

=====

TABLA 2: COMPARACIÓN CON LOS VALORES REPORTADOS EN EL ARTÍCULO

=====

Métrica	Artículo (Zhang et al.)	Nuestro resultado
Accuracy	0.97	0.981259
Precision	0.80	1.000000
Recall	0.66	0.549828
F1-Score	0.72	0.709534
RMSE (kW)	0.74	118.715234